EXPERIMENT 1

A. SHELL SCRIPT – ARITHMETIC OPERATION USING expr COMMAND

AIM: To perform basic arithmetic operations using the expr command in a shell script.

SOFTWARE REQUIRED: Oracle VirtualBox, Fedora Linux, Terminal, Bash shell.

PROCEDURE:

1. Open the Fedora terminal in Oracle VirtualBox.
2. Create a script using nano arithmetic.sh.
3. Enter the script and save the file.
4. Make it executable using chmod +x arithmetic.sh.
5. Run it using ./arithmetic.sh and enter two integers.

PROGRAM:

```
echo "Enter two integers:"
read a b
echo "Addition: $(expr "$a" + "$b")"
echo "Subtraction: $(expr "$a" - "$b")"
echo "Multiplication: $(expr "$a" \* "$b")"
if [ "$b" -ne 0 ]; then
   echo "Division: $(expr "$a" / "$b")"
else
   echo "Division is not possible (division by zero)."
Fi
```

OUTPUT:

```
Enter two integers:
12 4
Addition: 16
Subtraction: 8
Multiplication: 48
Division: 3
```

RESULT: Thus, the shell script performed arithmetic operations using the expr command.

EXPERIMENT 1

B.SHELL SCRIPT – CHECK LEAP YEAR USING if-else

AIM: To determine whether a given year is a leap year using if-else statements in a shell script.

SOFTWARE REQUIRED: Oracle VirtualBox, Fedora Linux, Terminal, Bash shell.

PROCEDURE:

1. Open the Fedora terminal and create leapyear.sh.
2. Type the program, save it, and make it executable with chmod +x leapyear.sh.
3. Run ./leapyear.sh and enter a year.

PROGRAM:

```
echo "Enter a year:"
read year
if [ $((year % 400)) -eq 0 ] || { [ $((year % 4)) -eq 0 ] && [ $((year % 100)) -ne 0 ]; };
then
   echo "$year is a leap year."
else
   echo "$year is not a leap year."
fi
```

OUTPUT:

```
Enter a year:
2024
2024 is a leap year.
```

RESULT: Thus, the script correctly checked whether the entered year was a leap year.

EXPERIMENT 2

A.SHELL SCRIPT – REVERSE A NUMBER USING while LOOP

AIM: To reverse the digits of an integer using a while loop in a shell script.

SOFTWARE REQUIRED: Oracle VirtualBox, Fedora Linux, Terminal, Bash shell.

PROCEDURE:

1. Create reverse.sh in the Fedora terminal.
2. Enter and save the script.
3. Run chmod +x reverse.sh followed by ./reverse.sh.
4. Enter a non-negative integer and observe the reversed number.

PROGRAM:

```
echo "Enter a number:"
read num
temp=$num
rev=0
while [ "$temp" -gt 0 ]; do
   digit=$(expr "$temp" % 10)
   rev=$(expr "$rev" \* 10 + "$digit")
   temp=$(expr "$temp" / 10)
done
echo "Reversed number: $rev"
```

OUTPUT:

```
Enter a number:
12345
Reversed number: 54321
```

RESULT: Thus, the script correctly checked whether the entered year was a leap year.

EXPERIMENT 2

B.SHELL SCRIPT – FIBONACCI SERIES USING for LOOP

AIM: To generate the Fibonacci series for a specified number of terms using a for loop.

SOFTWARE REQUIRED: Oracle VirtualBox, Fedora Linux, Terminal, Bash shell.

PROCEDURE:

1. Create fibonacci.sh.
2. Enter the program and save the file.
3. Make the file executable and run it.
4. Enter the required number of terms.

PROGRAM:

```
echo "Enter number of terms:"
read n
a=0
b=1
for ((i=0; i<n; i++)); do
   echo -n "$a "
   next=$((a + b))
   a=$b
   b=$next
done
echo
```

OUTPUT:

```
Enter number of terms:
8
0 1 1 2 3 5 8 13
```

RESULT: Thus, the Fibonacci series was generated successfully using a for loop.

EXPERIMENT 3

SYSTEM CALLS – fork(), exec(), getpid(), opendir(), readdir()

AIM: To demonstrate process creation, process identification, program execution, and directory handling using system/library calls in C.

SOFTWARE REQUIRED: Oracle VirtualBox, Fedora Linux, GCC compiler, Terminal.

PROCEDURE:

1. Start.
2. Include header files: Include stdio.h, stdlib.h and unistd.h.
3. Variable declaration: Declare an integer variable pid to store the return value of fork().
4. Create a process: Call fork() to create a child process.
5. If fork() returns -1, process creation fails.
6. If fork() returns 0, the current process is the child.
7. If fork() returns a positive value, the current process is the parent.
8. Print the first statement: Display THIS LINE EXECUTED TWICE. Both processes execute this statement.
9. Check process creation: If pid == -1, display CHILD PROCESS NOT CREATED and terminate the program.
10. Child process: If pid == 0, display the child's process ID using getpid() and its parent's process ID using getppid().
11. Parent process: If pid > 0, display the parent's process ID using getpid() and its parent process ID using getppid().
12. Print the final statement: Display IT CAN BE EXECUTED TWICE from both processes.
13. Stop.

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
```

```
#include <sys/wait.h>
#include <dirent.h>
int main(void)
{
   pid_t pid = fork();
   if (pid < 0)
   {
      perror("fork failed");
      return 1;
   }
   if (pid == 0)
   {
      printf("Child process PID: %d\n", getpid());
      printf("Executing ls using exec...\n");
      fflush(stdout);
      execlp("ls", "ls", "-1", (char *)NULL);
      perror("exec failed");
      exit(1);
   }
   else
   {
      printf("Parent process PID: %d\n", getpid());
      wait(NULL);
      DIR *dir = opendir(".");
      if (dir == NULL)
      {
         perror("opendir failed");
         return 1;
      }
      printf("\nDirectory contents:\n");
      struct dirent *entry;
      while ((entry = readdir(dir)) != NULL)
      {
         printf("%s\n", entry->d_name);
      }
      closedir(dir);
   }
   return 0;
}
```

OUTPUT:

Parent process PID: 4123

Child process PID: 4124

Executing ls using exec...

Desktop

Documents

Downloads

Music

Pictures

system_calls

system_calls.c

Directory contents:

.

..

Desktop

Documents

Downloads

Music

Pictures

system_calls

system_calls.c

RESULT: Thus, the fork(), execlp(), getpid(), opendir() and readdir() functions were implemented successfully to demonstrate process creation, program execution and directory handling.

EXPERIMENT 4

CPU SCHEDULING- FCFS, SJF, Priority, Round Robin

AIM:

To implement CPU scheduling algorithms, namely First Come First Serve (FCFS), Shortest Job First (SJF), Priority Scheduling, and Round Robin Scheduling, and calculate the waiting time and turnaround time of processes.

PROCEDURE:

1. Start the Fedora operating system using Oracle VirtualBox.
2. Open the terminal and create a Python program file.
3. Write the programs for the given CPU scheduling algorithms.
4. Execute each program using the Python 3 interpreter.
5. Enter the required process details, including burst time, priority, and time quantum where applicable.
6. Calculate and display the waiting time and turnaround time of each process.
7. Calculate the average waiting time and average turnaround time.
8. Verify the output and record the results.

PROGRAM:

(A) — First Come First Serve (FCFS)

```
bt = []

print("Enter the number of process:")
n = int(input())

print("Enter the burst time of the processes:")
bt = list(map(int, input().split()))

wt = []
tat = []
avgwt = 0
avgtat = 0

wt.insert(0, 0)
tat.insert(0, bt[0])

for i in range(1, len(bt)):
   wt.insert(i, wt[i-1] + bt[i-1])
```

```
    tat.insert(i, wt[i] + bt[i])

    avgwt += wt[i]
    avgtat += tat[i]

avgwt = float(avgwt) / n
avgtat = float(sum(tat)) / n

print("\n")
print("Process\tBurst Time\tWaiting Time\tTurnaround Time")

for i in range(0, n):
    print(str(i) + "\t\t" + str(bt[i]) +
          "\t\t" + str(wt[i]) + "\t\t" + str(tat[i]))

print("\nAverage Waiting time is:", avgwt)
print("Average Turn Around Time is:", avgtat)
```

OUTPUT:

Enter the number of process:

3

Enter the burst time of the processes:

24 3 3

Process Burst Time Waiting Time Turnaround Time

0 24 0 24

1 3 24 27

2 3 27 30

Average Waiting time is: 17.0

Average Turn Around Time is: 27.0

4(B) – SHORTEST JOB FIRST (SJF)

```
bt = []

print("Enter the number of processes:")
n = int(input())

print("Enter the burst time of the processes:")
```

```
bt = list(map(int, input().split()))

wt = [0] * n
tat = [0] * n

# Sort processes based on burst time
processes = sorted(enumerate(bt), key=lambda x: x[1])

avgwt = 0
avgtat = 0

for i in range(n):
    p, burst = processes[i]
    if i > 0:
        wt[i] = wt[i - 1] + processes[i - 1][1]
    tat[i] = wt[i] + burst
    avgwt += wt[i]
    avgtat += tat[i]

print("\nProcess\tBurst Time\tWaiting Time\tTurnaround Time")

for i in range(n):
    p, burst = processes[i]
    print(f"P{p + 1}\t{burst}\t\t{wt[i]}\t\t{tat[i]}")

print("\nAverage Waiting Time:", avgwt / n)
print("Average Turnaround Time:", avgtat / n)
```

OUTOUT:

```
Enter the number of processes:
3
Enter the burst time of the processes:
6 8 2

Process  Burst Time  Waiting Time  Turnaround Time
P3       2          0             2
P1       6          2             8
P2       8          8             16

Average Waiting Time: 3.3333333333333335
Average Turnaround Time: 8.666666666666666
```

4(C) – PRIORITY

```
n = int(input("Enter the number of processes: "))

processes = []
```

```
for i in range(n):
    bt = int(input(f"Enter burst time for P{i+1}: "))
    priority = int(input(f"Enter priority for P{i+1}: "))
    processes.append([i+1, bt, priority])

# Lower priority number means higher priority
processes.sort(key=lambda x: x[2])

wt = [0] * n
tat = [0] * n

for i in range(1, n):
    wt[i] = wt[i-1] + processes[i-1][1]

for i in range(n):
    tat[i] = wt[i] + processes[i][1]

print("\nProcess\tBurst\tPriority\tWaiting\tTurnaround")

for i in range(n):
    print(f"P{processes[i][0]}\t{processes[i][1]}\t{processes[i][2]}\t\t{wt[i]}\t{tat[i]}")

print("\nAverage Waiting Time:", sum(wt) / n)
print("Average Turnaround Time:", sum(tat) / n)
```

OUTPUT:

```
Enter the number of processes: 3
Enter burst time for P1: 6
Enter priority for P1: 2
Enter burst time for P2: 8
Enter priority for P2: 3
Enter burst time for P3: 2
Enter priority for P3: 1

Process  Burst  Priority  Waiting  Turnaround
P3       2      1         0        2
P1       6      2         2        8
P2       8      3         8        16

Average Waiting Time: 3.3333333333333335
Average Turnaround Time: 8.666666666666666
```

4(D) – ROUND ROBIN

```
n = int(input("Enter the number of processes: "))
```

```
bt = list(map(int, input("Enter burst times: ").split()))
tq = int(input("Enter time quantum: "))

rem_bt = bt.copy()
wt = [0] * n
tat = [0] * n
time = 0

while True:
    done = True

    for i in range(n):
        if rem_bt[i] > 0:
            done = False

            if rem_bt[i] > tq:
                time += tq
                rem_bt[i] -= tq
            else:
                time += rem_bt[i]
                wt[i] = time - bt[i]
                rem_bt[i] = 0

    if done:
        break

for i in range(n):
    tat[i] = bt[i] + wt[i]

print("\nProcess\tBurst\tWaiting\tTurnaround")

for i in range(n):
    print(f"P{i+1}\t{bt[i]}\t{wt[i]}\t{tat[i]}")

print("\nAverage Waiting Time:", sum(wt) / n)
print("Average Turnaround Time:", sum(tat) / n)
```

OUTPUT:

```
Enter the number of processes: 3
Enter burst times: 5 4 2
Enter time quantum: 2

Process  Burst  Waiting  Turnaround
P1       5      6        11
P2       4      6        10
```

P3 2 4 6

Average Waiting Time: 5.333333333333333
Average Turnaround Time: 9.0
RESULT:

Thus, the CPU scheduling algorithms FCFS, SJF, Priority Scheduling, and Round Robin were implemented and executed successfully, and the waiting time, turnaround time, and their averages were calculated.

EXPERIMENT 5

IPC USING SHARED MEMORY

AIM: To write a C program to do Inter Process Communication (IPC) using shared memory between sender process and receiver process.

SOFTWARE REQUIRED: Oracle VirtualBox, Fedora Linux, GCC compiler, Terminal.

ALGORITHM:

SENDER:

1. Set the size of the shared memory segment
2. Allocate the shared memory segment using shmget
3. Attach the shared memory segment using shmat
4. Write a string to the shared memory segment using sprintf
5. Set delay using sleep
6. Detach shared memory segment using shmdt

RECIEVER:

1. Set the size of the shared memory segment
2. Allocate the shared memory segment using shmget
3. Attach the shared memory segment using shmat
4. Print the shared memory contents sent by the sender

process.

5. Detach shared memory segment using shmdt

PROGRAM:

(A) SENDER.C

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ipc.h>
#include <sys/shm.h>

int main()
{
   int shmid;
   char *str;

   shmid = shmget(1234, 1024, 0666 | IPC_CREAT);

   if (shmid == -1)
   {
      perror("shmget");
      return 1;
   }

   str = (char *)shmat(shmid, NULL, 0);

   if (str == (char *)-1)
   {
      perror("shmat");
      return 1;
   }

   printf("Enter the message: ");
   fgets(str, 1024, stdin);

   printf("Message sent: %s", str);

   shmdt(str);

   return 0;
}
```

(B) RECEIVER.C

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/ipc.h>
```

```
#include <sys/shm.h>

int main()
{
   int shmid;
   char *str;

   shmid = shmget(1234, 1024, 0666);

   if (shmid == -1)
   {
      perror("shmget");
      return 1;
   }

   str = (char *)shmat(shmid, NULL, 0);

   if (str == (char *)-1)
   {
      perror("shmat");
      return 1;
   }

   printf("Message received: %s", str);

   shmdt(str);
   shmctl(shmid, IPC_RMID, NULL);

   return 0;
}
```

OUTPUT:

Enter the message: Hello from Sender

Message sent: Hello from Sender

Message received: Hello from Sender

RESULT:

Thus, Inter-Process Communication using shared memory was implemented and executed successfully in the Fedora Linux operating system.

EXPERIMENT 6

PRODUCER–CONSUMER PROBLEM USING SEMAPHORES

AIM: To implement the Producer–Consumer problem using semaphores for process synchronization in the Linux operating system.

SOFTWARE REQUIRED: Oracle VirtualBox, Fedora Linux, GCC compiler, Terminal.

PROCEDURE:

1. Start the Fedora operating system using Oracle VirtualBox.
2. Open the terminal and create a C program.
3. Include the required header files for threads and semaphores.
4. Initialize the empty and full semaphores.
5. Create producer and consumer threads.
6. The producer inserts items into the shared buffer.
7. The consumer removes items from the shared buffer.
8. Use semaphores to prevent buffer overflow and underflow, and a mutex to protect buffer access.
9. Display the produced and consumed items.
10. Compile, execute and verify the output.

PROGRAM:

```
#include<stdio.h>
#include<stdlib.h>

int mutex=1,full=0,empty=3,x=0;

void producer();
void consumer();
int wait(int);
int signal(int);

int main()
{
   int n;

   printf("\n1.Producer\n2.Consumer\n3.Exit");

   while(1)
   {
      printf("\nEnter your choice:");
      scanf("%d",&n);
```

```
        switch(n)
        {
            case 1:
                if((mutex==1)&&(empty!=0))
                    producer();
                else
                    printf("Buffer is full!!");
                break;

            case 2:
                if((mutex==1)&&(full!=0))
                    consumer();
                else
                    printf("Buffer is empty!!");
                break;

            case 3:
                exit(0);
                break;
        }
    }

    return 0;
}

int wait(int s)
{
    return (--s);
}

int signal(int s)
{
    return (++s);
}

void producer()
{
    mutex=wait(mutex);
    full=signal(full);
    empty=wait(empty);
    x++;

    printf("\nProducer produces the item %d",x);

    mutex=signal(mutex);
}
```

```
void consumer()
{
   mutex=wait(mutex);
   full=wait(full);
   empty=signal(empty);

   printf("\nConsumer consumes item %d",x);
   x--;

   mutex=signal(mutex);
}
```

OUTPUT:

1.Producer

2.Consumer

3.Exit

Enter your choice:1

Producer produces the item 1

Enter your choice:1

Producer produces the item 2

Enter your choice:1

Producer produces the item 3

Enter your choice:1

Buffer is full!!

Enter your choice:2

Consumer consumes item 3

Enter your choice:2

Consumer consumes item 2

Enter your choice:2

Consumer consumes item 1

Enter your choice:2

Buffer is empty!!

Enter your choice:3

RESULT:

Thus, the Producer–Consumer problem was implemented and executed successfully using simulated semaphore operations in C.

EXPERIMENT 7

BANKER'S DEADLOCK AVOIDANCE ALGORITHM

AIM:

To implement the Banker's Algorithm for deadlock avoidance and determine whether the system is in a safe state.

PROCEDURE:

1. Initialize work=available and finish[i]=false for all values of i

2.Find an i such that both:

finish[i]=false and Needi<= work

3. If no such i exists go to step 6

4. Compute work=work+allocationi

5. Assign finish[i] to true and go to step 2

6. If finish[i]==true for all i, then print safe sequence

7. Else print there is no safe sequence

PROGRAM:

```
#include <stdio.h>
int main()
{
   // P0, P1, P2, P3, P4 are process names
   int n, m, i, j, k;
   n = 5; // Number of processes
   m = 3; // Number of resources
   // Allocation Matrix
   int alloc[5][3] = {
      {0, 1, 0},
      {2, 0, 0},
      {3, 0, 2},
      {2, 1, 1},
      {0, 0, 2}
   };

   // Maximum Matrix
```

```
int max[5][3] = {
   {7, 5, 3},
   {3, 2, 2},
   {9, 0, 2},
   {2, 2, 2},
   {4, 3, 3}
};

// Available Resources
int avail[3] = {3, 3, 2};

int f[n], ans[n], ind = 0;

for(k = 0; k < n; k++)
{
   f[k] = 0;
}

int need[n][m];

for(i = 0; i < n; i++)
{
   for(j = 0; j < m; j++)
      need[i][j] = max[i][j] - alloc[i][j];
}

int y = 0;

for(k = 0; k < 5; k++)
{
   for(i = 0; i < n; i++)
   {
      if(f[i] == 0)
      {
         int flag = 0;

         for(j = 0; j < m; j++)
         {
            if(need[i][j] > avail[j])
            {
               flag = 1;
               break;
            }
         }

         if(flag == 0)
         {
```

```
                ans[ind++] = i;

                for(y = 0; y < m; y++)
                   avail[y] += alloc[i][y];

                f[i] = 1;
             }
          }
       }
    }

    printf("The SAFE Sequence is \n");

    for(i = 0; i < n - 1; i++)
       printf(" P%d ->", ans[i]);

    printf(" P%d", ans[n - 1]);

    return (0);
}
```

OUTPUT:

The SAFE Sequence is

P1 -> P3 -> P4 -> P0 -> P2

RESULT:

Thus, the Banker's Algorithm for deadlock avoidance was implemented successfully, and the system was found to be in a safe state.

EXPERIMENT 8

Contiguous Memory Allocation: BEST FIT

AIM:

To implement the Best Fit memory allocation algorithm and allocate memory blocks to processes efficiently.

PROCEDURE:

1. Start the Fedora operating system using Oracle VirtualBox.
2. Open the terminal and create a C program.
3. Enter the number of memory blocks and processes.
4. Read the sizes of memory blocks and processes.
5. For each process, find the smallest available memory block that can accommodate it.
6. Allocate the process to the selected block and update the remaining block size.
7. Repeat until all processes have been considered.
8. Display the allocation details and verify the output.

PROGRAM:

```
#include <stdio.h>
int main()
{
   int b[20], p[20], allocation[20];
   int nb, np, i, j, best;
   printf("Enter number of memory blocks: ");
   scanf("%d", &nb);
   printf("Enter memory block sizes:\n");
   for(i = 0; i < nb; i++)
      scanf("%d", &b[i]);
   printf("Enter number of processes: ");
   scanf("%d", &np);
   printf("Enter process sizes:\n");
   for(i = 0; i < np; i++)
      scanf("%d", &p[i]);

   for(i = 0; i < np; i++)
      allocation[i] = -1;

   for(i = 0; i < np; i++)
   {
      best = -1;
```

```
        for(j = 0; j < nb; j++)
        {
            if(b[j] >= p[i])
            {
                if(best == -1 || b[j] < b[best])
                    best = j;
            }
        }

        if(best != -1)
        {
            allocation[i] = best;
            b[best] -= p[i];
        }
    }

    printf("\nProcess\tSize\tBlock Allocated\n");

    for(i = 0; i < np; i++)
    {
        printf("P%d\t%d\t", i+1, p[i]);

        if(allocation[i] != -1)
            printf("%d\n", allocation[i]+1);
        else
            printf("Not Allocated\n");
    }

    return 0;
}
```

OUTPUT:

Enter number of blocks: 5

Enter block sizes:

100 500 200 300 600

Enter number of processes: 4

Enter process sizes:

212 417 112 426

Process Size Block

P1 212 4

P2 417 2

P3 112 3

P4 426 5

RESULT:

Thus, the Best Fit memory allocation algorithm was implemented and executed successfully in Fedora Linux, and memory blocks were allocated to the processes according to their requirements.

EXPERIMENT 9

Contiguous Memory Allocation: FIRST FIT

AIM:

To implement the First Fit memory allocation algorithm in C

PROCEDURE:

1. Start Fedora Linux using Oracle VirtualBox.
2. Open the terminal and create a C program.
3. Enter the number of memory blocks and processes.
4. Read the memory block sizes and process sizes.
5. Check each memory block sequentially for every process.
6. Allocate the process to the first block having sufficient space.
7. Update the remaining size of the allocated block.
8. Display the memory allocation details.
9. Stop.

PROGRAM:

```
#include <stdio.h>

int main()
{
   int b[20], p[20], a[20];
   int nb, np, i, j;

   printf("Enter number of blocks: ");
   scanf("%d", &nb);

   printf("Enter block sizes:\n");
   for(i = 0; i < nb; i++)
      scanf("%d", &b[i]);

   printf("Enter number of processes: ");
   scanf("%d", &np);

   printf("Enter process sizes:\n");
   for(i = 0; i < np; i++)
      scanf("%d", &p[i]);

   for(i = 0; i < np; i++)
   {
      a[i] = -1;
```

```
        for(j = 0; j < nb; j++)
        {
            if(b[j] >= p[i])
            {
                a[i] = j;
                b[j] -= p[i];
                break;
            }
        }
    }

    printf("\nProcess\tSize\tBlock\n");

    for(i = 0; i < np; i++)
    {
        printf("P%d\t%d\t", i+1, p[i]);

        if(a[i] == -1)
            printf("Not Allocated\n");
        else
            printf("%d\n", a[i]+1);
    }

    return 0;
}
```

OUTPUT:

Enter number of blocks: 5

Enter block sizes:

100 500 200 300 600

Enter number of processes: 4

Enter process sizes:

212 417 112 426

Process Size Block

P1 212 2

P2 417 5

P3 112 2

P4 426 Not Allocated

RESULT:

Thus, the First Fit memory allocation algorithm was implemented and executed successfully using C in Fedora Linux.

EXPERIMENT 10

Non-Contiguous Memory Allocation: FIFO

AIM:

To implement the First In First Out (FIFO) page replacement algorithm and calculate the total number of page faults.

PROCEDURE:

1. Declare the size with respect to page length

2. Check the need of replacement from the page to memory

3. Check the need of replacement from old page to new page in memory 4.

Form a queue to hold all pages

5. Insert the page require memory into the queue

6. Check for bad replacement and page fault

7. Get the number of processes to be inserted

8. Display the values

PROGRAM:

```
def fifo():
    global a, n, m
    f = -1
    page_faults = 0
    page = []

    for i in range(m):
        page.append(-1)

    for i in range(n):
        flag = 0

        for j in range(m):
            if page[j] == a[i]:
                flag = 1
                break

        if flag == 0:
            f = (f + 1) % m
```

```
            page[f] = a[i]
            page_faults += 1

            print("\n%d ->" % a[i], end=" ")

            for j in range(m):
                if page[j] != -1:
                    print(page[j], end=" ")
                else:
                    print("-", end=" ")
        else:
            print("\n%d -> No Page Fault" % a[i], end=" ")

    print("\n Total page faults : %d." % page_faults)


a = []
n = int(input("\n Enter the size of reference string : "))

for i in range(n):
    a.append(int(input(" Enter [%2d] : " % (i + 1))))

m = int(input("\n Enter page frame size : "))

fifo()
```

OUTPUT:

Enter the size of reference string : 8

Enter [1] : 7

Enter [2] : 0

Enter [3] : 1

Enter [4] : 2

Enter [5] : 0

Enter [6] : 3

Enter [7] : 0

Enter [8] : 4

Enter page frame size : 3

7 -> 7 - -

0 -> 7 0 -

1 -> 7 0 1

2 -> 2 0 1

0 -> No Page Fault

3 -> 2 3 1

0 -> 2 3 0

4 -> 4 3 0

Total page faults : 7.

RESULT:

Thus, the First In First Out (FIFO) page replacement algorithm was implemented and executed successfully using Python in Fedora Linux, and the total number of page faults was calculated.

EXPERIMENT 11

Non-Contiguous Memory Allocation: LRU

AIM:

To implement the Least Recently Used (LRU) page replacement algorithm and calculate the total number of page faults.

PROCEDURE:

1: Start the process

2: Declare the size

3: Get the number of pages to be inserted

4: Get the value

5: Declare counter and stack

6: Select the least recently used page by counter value

7: Stack them according the selection.

8: Display the values

9: Stop the process

PROGRAM:

```
#include <stdio.h>

int main()
{
   int frames[10], pages[30], time[10];
   int n, m, i, j, pos, faults = 0, count = 0;
   int found, min;

   printf("Enter number of frames: ");
   scanf("%d", &m);

   printf("Enter number of pages: ");
   scanf("%d", &n);

   printf("Enter reference string: ");
   for(i = 0; i < n; i++)
```

```
    scanf("%d", &pages[i]);

for(i = 0; i < m; i++)
{
    frames[i] = -1;
    time[i] = 0;
}

for(i = 0; i < n; i++)
{
    found = 0;

    for(j = 0; j < m; j++)
    {
        if(frames[j] == pages[i])
        {
            found = 1;
            time[j] = ++count;
            break;
        }
    }

    if(found == 0)
    {
        faults++;

        pos = -1;

        for(j = 0; j < m; j++)
        {
            if(frames[j] == -1)
            {
                pos = j;
                break;
            }
        }

        if(pos == -1)
        {
            pos = 0;
            min = time[0];

            for(j = 1; j < m; j++)
            {
                if(time[j] < min)
                {
                    min = time[j];
```

```
                pos = j;
            }
          }
        }

        frames[pos] = pages[i];
        time[pos] = ++count;
      }

      printf("\n");
      for(j = 0; j < m; j++)
        printf("%d\t", frames[j]);
    }

    printf("\n\nTotal Page Faults = %d\n", faults);

    return 0;
}
```

OUTPUT:

Enter number of frames: 3

Enter number of pages: 8

Enter reference string: 7 0 1 2 0 3 0 4

7 -1 -1

7 0 -1

7 0 1

2 0 1

2 0 1

2 0 3

2 0 3

4 0 3

Total Page Faults = 6

RESULT:

Thus, the Least Recently Used (LRU) page replacement algorithm was implemented successfully using C in Fedora Linux, and the total number of page faults was calculated.

EXPERIMENT 12

Non-Contiguous Memory Allocation: OPTIMAL

AIM:

To implement the Optimal Page Replacement algorithm in C and calculate the total number of page faults.

PROCEDURE:

1.Start the process

2.Declare the size

3.Get the number of pages to be inserted

4.Get the value

5.Declare counter and stack

6.Select the least frequently used page by counter value

7.Stack them according the selection.

8.Display the values

9.Stop the process

PROGRAM:

```
#include <stdio.h>
int main()
{
   int frames[10], pages[30];
   int n, m, i, j, k, pos, faults = 0;
   int found, farthest, next;

   printf("Enter number of frames: ");
   scanf("%d", &m);

   printf("Enter number of pages: ");
   scanf("%d", &n);

   if(m < 1 || m > 10 || n < 1 || n > 30)
   {
      printf("Invalid input size\n");
      return 1;
```

```
}

printf("Enter reference string: ");
for(i = 0; i < n; i++)
   scanf("%d", &pages[i]);

for(i = 0; i < m; i++)
   frames[i] = -1;

for(i = 0; i < n; i++)
{
   found = 0;

   for(j = 0; j < m; j++)
   {
      if(frames[j] == pages[i])
      {
         found = 1;
         break;
      }
   }

   if(found == 0)
   {
      faults++;
      pos = -1;

      for(j = 0; j < m; j++)
      {
         if(frames[j] == -1)
         {
            pos = j;
            break;
         }
      }

      if(pos == -1)
      {
         farthest = -1;

         for(j = 0; j < m; j++)
         {
            for(k = i + 1; k < n; k++)
            {
               if(frames[j] == pages[k])
                  break;
            }
```

```
                next = k;

                if(next > farthest)
                {
                   farthest = next;
                   pos = j;
                }
             }
          }

          frames[pos] = pages[i];
       }

       printf("\n");
       for(j = 0; j < m; j++)
          printf("%d\t", frames[j]);
    }

    printf("\n\nTotal Page Faults = %d\n", faults);

    return 0;
}
```

OUTPUT:

Enter number of frames: 3

Enter number of pages: 8

Enter reference string: 7 0 1 2 0 3 0 4

7 -1 -1

7 0 -1

7 0 1

2 0 1

2 0 1

3 0 1

3 0 1

4 0 1

Total Page Faults = 6

RESULT:

Thus, the Optimal Page Replacement algorithm was implemented and executed successfully using C in Fedora Linux, and the total number of page faults was calculated.